FR. CONCEICAO RODRIGUES COLLEGE OF ENGINEERING

Department of Computer Engineering
Academic Year 2025-26

Rubrics for Lab Experiments

Class : B.*E. Computer Engineering*
Subject Name :*DC Lab*
Semester : VIII
Subject Code : *CSL801*

Practical No	2
Title	Implementing arithmetic operations using RPC
Date of Performance	11/11/25
Roll No	9913
Student Name	Mark Lopes

Evaluation

Performance Indicator	Below average	Average	Good	Excellent	Marks
On time Submission (2)	Not submitted (0)	Submitted after deadline (1)	Early or on time submission(2)	---	
Test cases and output (4)	Incorr ect output (1)	The expected output is verified only a for few test cases (2)	The expected output is Verified for all test cases but is not presentable (3)	Expected output is obtained for all test cases. Presentable and easy to follow (4)	
Coding efficiency (2)	The code is not structured at all (0)	The code is structured but not efficient (1)	The code is Structured and efficient. (2)	-	
Knowledge(2)	Basic concepts not clear (0)	Understood the basic concepts (1)	Could explain the concept with suitable example (1.5)	Could relate the theory with real world application(2)	
Total					

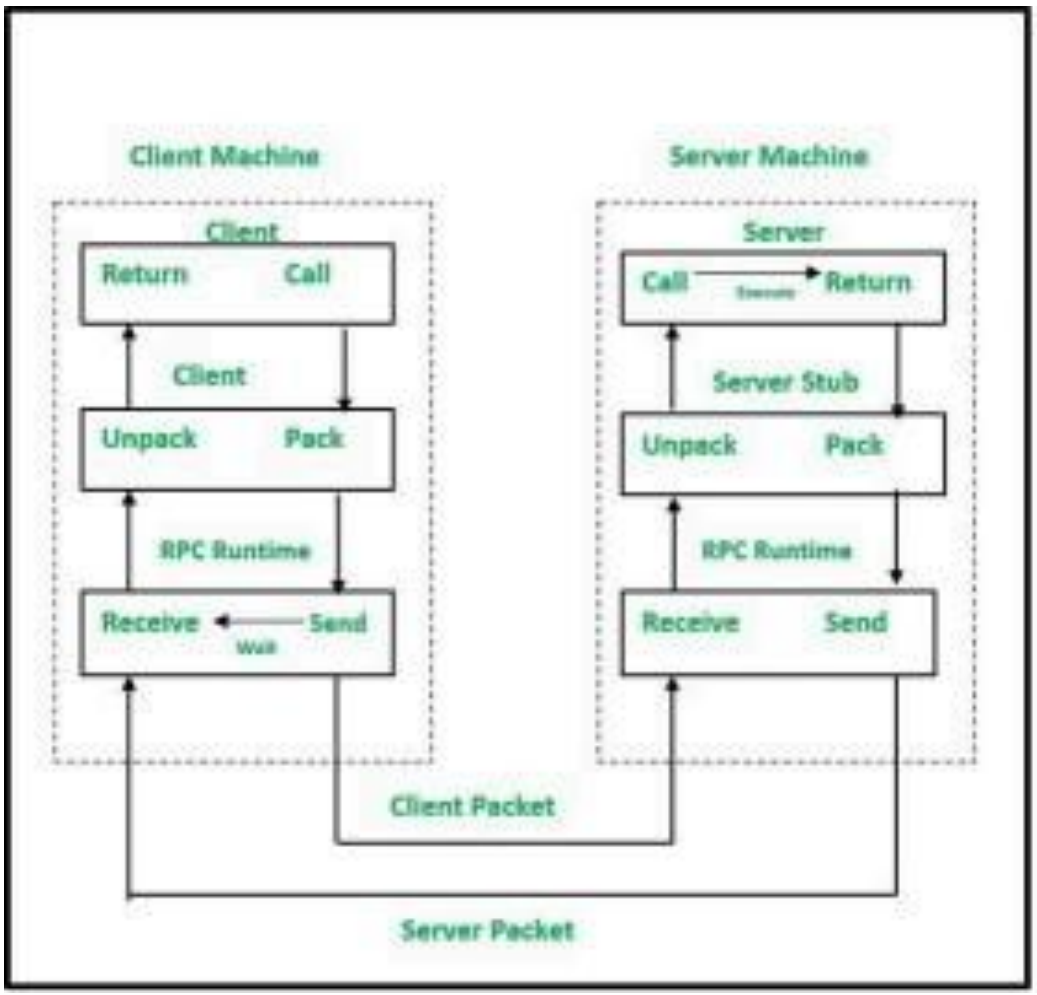
Signature of the Teacher:

Theory:

RPC is an effective mechanism for building client-server systems that are distributed. RPC enhances the power and ease of programming of the client/server computing concept. It is a protocol that allows one software to seek a service from another program on another computer in a network without having to know about the network. The software that makes the request is called a client, and the program that provides the service is called a server.

There are 5 elements used in the working of RPC:

- Client
- Client Stub
- RPC Runtime
- Server Stub
- Server



- The client, the client stub, and one instance of RPC Runtime are all running on the client machine.

- A client initiates a client stub process by giving parameters as normal. The client stub acquires storage in the address space of the client.

- At this point, the user can access RPC by using a normal Local Procedural Call. The RPC runtime oversees message transmission between client and server via the network. Retransmission, acknowledgment, routing, and encryption are all tasks performed by it.

- On the server-side, values are returned to the server stub, after the completion of server operation, which then packs (which is also known as marshalling) the return values into a message. The transport layer receives a message from the server stub.

- The resulting message is transmitted by the transport layer to the client transport layer, which then sends a message back to the client stub.

- The client stub unpacks (which is also known as unmarshalling) the return arguments in the resulting packet, and the execution process returns to the caller at this point.

Calculator.java

```
package exp2;

public interface Calculator {
    double add(double a, double b);
    double sub(double a, double b);
    double mul(double a, double b);
    double div(double a, double b) throws ArithmeticException;
}
```

—----------

CalculatorImpl.java

```
package exp2;
public class CalculatorImpl implements Calculator {

    @Override
    public double add(double a, double b) {
        return a + b;
    }

    @Override
    public double sub(double a, double b) {
        return a - b;
    }

    @Override
    public double mul(double a, double b) {
        return a * b;
    }

    @Override
    public double div(double a, double b) throws ArithmeticException
{
        if (b == 0) throw new ArithmeticException("Division by zero");
        return a / b;
```

```
    }
}
<-------------------------------------------------------------------------------->
```

RPCServer.java

```
package exp2;

import java.io.*;
import java.net.*;

public class RPCServer {

    private final int port;
    private final CalculatorImpl calc = new CalculatorImpl();

    public RPCServer(int port) {
        this.port = port;
    }

    public void start() throws IOException {
        ServerSocket serverSocket = new ServerSocket(port);
        System.out.println("Shwen's RPC Server started on port " +
port);

        while (true) {
            Socket clientSocket = serverSocket.accept();
            new Thread(new ClientHandler(clientSocket, calc)).start();
        }
    }

    public static void main(String[] args) {
        int port = 5000;

        if (args.length >= 1) {
            try {
                port = Integer.parseInt(args[0]);
```

```
            } catch (NumberFormatException ignored) {}
        }

        try {
            new RPCServer(port).start();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private static class ClientHandler implements Runnable {

        private final Socket socket;
        private final CalculatorImpl calc;

        public ClientHandler(Socket socket, CalculatorImpl calc) {
            this.socket = socket;
            this.calc = calc;
        }

        @Override
        public void run() {
            try (
                BufferedReader in = new BufferedReader(
                    new InputStreamReader(socket.getInputStream())
                );
                PrintWriter out = new
PrintWriter(socket.getOutputStream(), true)
            ) {
                // Read one request line (protocol: METHOD arg1 arg2)
                String request = in.readLine();
                if (request == null) return;

                String resp = handleRequest(request.trim());
                out.println(resp);
```

```
            } catch (IOException e) {
                e.printStackTrace();
            } finally {
                try {
                    socket.close();
                } catch (IOException ignored) {}
            }
        }

        private String handleRequest(String req) {
            // Very simple protocol: METHOD arg1 arg2
            String[] parts = req.split("\\s+");

            if (parts.length < 3) {
                return "ERR Invalid request. Use: ADD|SUB|MUL|DIV
<num1> <num2>";
            }

            String method = parts[0].toUpperCase();
            double a, b;

            try {
                a = Double.parseDouble(parts[1]);
                b = Double.parseDouble(parts[2]);
            } catch (NumberFormatException e) {
                return "ERR Number format error";
            }

            try {
                double result;

                switch (method) {
                    case "ADD": result = calc.add(a, b); break;
                    case "SUB": result = calc.sub(a, b); break;
```

```
                case "MUL": result = calc.mul(a, b); break;
                case "DIV": result = calc.div(a, b); break;
                default: return "ERR Unknown method: " + method;
            }

            return "OK " + result;

        } catch (ArithmeticException ae) {
            return "ERR " + ae.getMessage();
        } catch (Exception e) {
            return "ERR Internal server error";
        }
    }
  }
}
```

<---------->

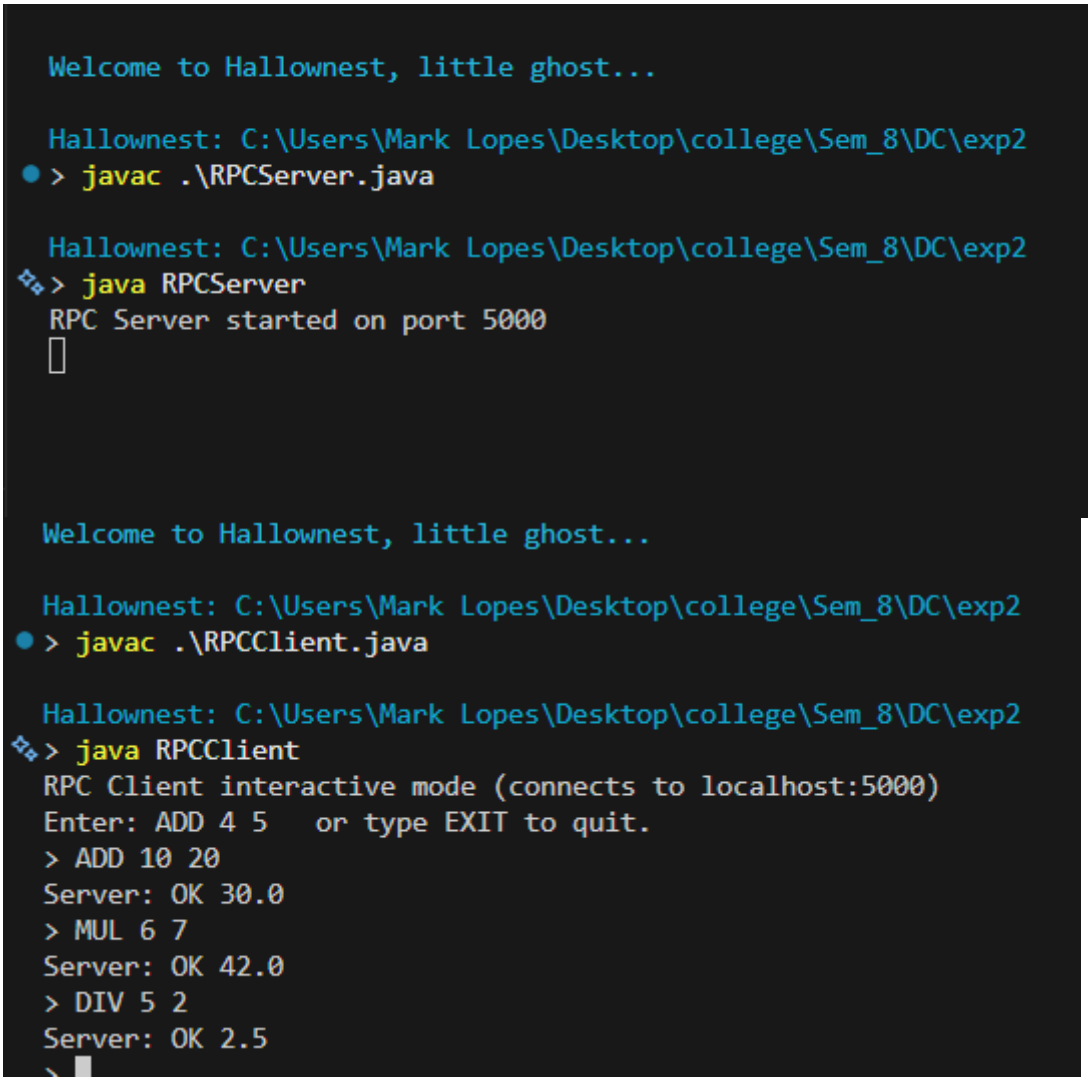
RPCClient.java

```
package exp2;

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class RPCClient {

    private final String host;
    private final int port;

    public RPCClient(String host, int port) {
        this.host = host;
        this.port = port;
    }

    // Sends a single request and returns server response
```

```
    public String sendRequest(String request) throws IOException {
        try (
            Socket socket = new Socket(host, port);
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream())
            );
            PrintWriter out = new PrintWriter(socket.getOutputStream(),
true)
        ) {
            out.println(request);
            return in.readLine();
        }
    }

    public static void main(String[] args) {

        String host = "localhost";
        int port = 5000;

        if (args.length >= 1) host = args[0];
        if (args.length >= 2) {
            try { port = Integer.parseInt(args[1]); }
            catch (NumberFormatException ignored) {}
        }

        RPCClient client = new RPCClient(host, port);

        // CLI quick mode: java RPCClient host port METHOD n1 n2
        if (args.length >= 5) {
            String method = args[2];
            String n1 = args[3];
            String n2 = args[4];
            String req = method + " " + n1 + " " + n2;

            try {
```

```
            String resp = client.sendRequest(req);
            System.out.println("Server response: " + resp);
        } catch (IOException e) {
            System.err.println("Failed to send request: " +
e.getMessage());
        }
        return;
    }

    // Interactive mode
    System.out.println("RPC Client interactive mode (connects to "
+ host + ":" + port + ")");
    System.out.println("Enter requests like: ADD 4 5 or type EXIT
to quit.");

    Scanner sc = new Scanner(System.in);

    while (true) {
        System.out.print("> ");
        String line = sc.nextLine().trim();

        if (line.equalsIgnoreCase("EXIT") ||
line.equalsIgnoreCase("QUIT")) break;
        if (line.isEmpty()) continue;

        try {
            String resp = client.sendRequest(line);
            System.out.println("Server: " + resp);
        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    sc.close();
    System.out.println("Client exiting.");
```

```
    }
}
```



Conclusion :

This experiment demonstrates how Remote Procedure Call (RPC) enables communication between a client and a server as if they were executing local functions. By defining procedures in an interface file and using rpcgen, we successfully implemented arithmetic operations that are executed remotely. The client sends requests, the server performs the computation, and the result is returned transparently. This shows that RPC simplifies distributed computing by hiding the details of network communication and allowing processes on different systems to interact seamlessly

Postlab :

1. In which category of communication can RPC be included?
RPC falls under synchronous communication, because the client sends a request and then waits for the server's response before continuing execution.

2. What are stubs? What are the different ways of stub generation?
Stubs are client-side proxy objects that represent remote objects in RPC or RMI systems. They hide the underlying communication by forwarding method calls to the remote server and by performing marshalling and unmarshalling of parameters and results. Stubs can be generated in two ways: static generation, which is done using tools such as the *rmic* compiler, and dynamic generation, which happens automatically at runtime without requiring manual stub creation.

3. What is binding?
Binding is the process of associating a name with a remote object in the RMI registry so that clients can look it up and access it. Through binding, clients can locate and invoke methods on remote objects simply by using a known and registered name, which makes interaction with distributed components straightforward.

4. Name the transparencies achieved through stubs.
Stubs provide several important transparencies in distributed systems. They provide access transparency by allowing clients to access remote objects as though they were local. They also provide location transparency by ensuring the client does not need to know where the remote object is physically located. Lastly, they offer communication transparency by hiding low-level details of message passing, marshalling, and unmarshalling from the user.